Faculty of Technology, Design & Environment
School of Engineering, Computing & Mathematics

OXFORD BROOKES UNIVERSITY

Assessment cover

STUDENTS, PLEASE COPY THIS PAGE AND USE AS THE COVER FOR YOUR SUBMISSION

Module No:	COMP6011	Module title:	Machine Learning

Assessment number:	**CW2**	Assessment title:	**Machine Learning Algorithm Implementation and Evaluation**

Banner assignment identifier	*CWS1WEEK12*	Due date and time**:**	**Dec 12, 2025, 1pm**

Estimated total time to be spent on assignment:	*20 hours per student direct work on coursework, plus 22 hours per student on independent study*

LEARNING OUTCOMES

On successful completion of this assignment, students will be able to achieve the following learning outcomes (LOs): *LO numbers and text to be copied and pasted from the module handbook*
Critically evaluate the requirements for application of system intelligence in an autonomous system.
Build independence, confidence and advanced professional awareness skills within the discipline including ethical, social, legal and risk considerations.
Synthesise and independently develop complex documentation to support autonomous systems created.

Statement of Compliance
By submitting this assessment I declare that the work submitted is my own and that the work I submit is fully in accordance with the University regulations regarding assessments. (*www.brookes.ac.uk/uniregulations/current*)

Use of AI Tools: You are required to use this form to declare which AI tools you have used and how you have used them. Please complete the form and attach it to your submission as an Appendix, if you have used such tools.

❖ Part A: Explanation of Binary Classification

In this coursework we work with the ***Heart Failure Clinical Records*** dataset, which contains several clinical features describing patients and a target variable named "**death_event"**. This variable indicates if the patient died during follow-up, so the problem naturally becomes a **binary classification task**. As we saw in *Lecture 1 (Introduction to Machine Learning)*, binary classification deals with predicting one of two possible outcomes based on labelled training examples.

The main goal of the model is to learn a function that maps patient characteristics to a probability of death. During the lectures, especially in *Lecture 6 and Lecture 7*, we studied how neural networks can approximate non-linear decision boundaries by combining multiple layers of neurons. Implementing the algorithms manually in this coursework helped me to understand how the forward pass, activation functions and backpropagation work behind the scenes. For instance, the sigmoid function that we used in all layers is the same one introduced on slide 12 of *Lecture 6*, which compresses outputs into the $[0, 1]$interval and allows us to interpret them as probabilities.

Since the dataset comes from a medical context, the consequences of misclassification are important. As we discussed briefly in class, false negatives (predicting that the patient survives when they actually die) are more harmful than false positives. This means that, besides accuracy, other evaluation methods such as a confusion matrix are essential to interpret the final performance. This idea appears also in external literature, where authors emphasise the calenes of predicting mortality using clinical data.

In summary, binary classification is an appropriate framework for this task, and the coursework provides a good opportunity to put into practice the theoretical principles introduced in the lectures, including supervised learning, loss functions, activation functions and network optimisation.

❖ Part B: Implementation and First Experiments

This part involved building a simple neural network from scratch using NumPy. The implementation follows the structure shown in *Lecture 6 and Lecture 7*, with a forward pass that computes activations layer by layer and a backward pass that propagates the error using derivatives of the sigmoid function.

▪ Part B.a: Implementing a Single-Layer Perceptron

Following the structure from *Lecture 6 and Lecture 7*, I implemented a simple neural network with one hidden layer. The forward pass applies the sigmoid activation to each neuron, computing

$$A=\sigma(WX+b)$$

as shown in slide 14 of *Lecture 6*.

For the backward pass, I followed the chain-rule equations presented in class. Backpropagation was described on *Lecture 7*, and implementing it manually helped me understand how the gradients move from the output back through each layer. Although the maths were challenging, rewriting the equations in code made the process much clearer.

The optimisation method used was batch gradient descent. According to Rumelhart, Hinton & Williams (1986), this algorithm updates weights in the direction that minimises the loss function. Here we applied binary cross-entropy, also introduced in class.

- **Part B.b: Training and Evaluation of the Perceptron**

After implementing the perceptron, I trained it on the training set and evaluated accuracy on the validation set. The model reached approximately 0.70 validation accuracy, which is comparable to the results we saw in the lecture examples when using simple architectures and noisy datasets.

The learning curves showed a smooth decrease in training loss and a relatively stable validation loss, similar to the patterns seen in slide 8 of *Lecture 7*.

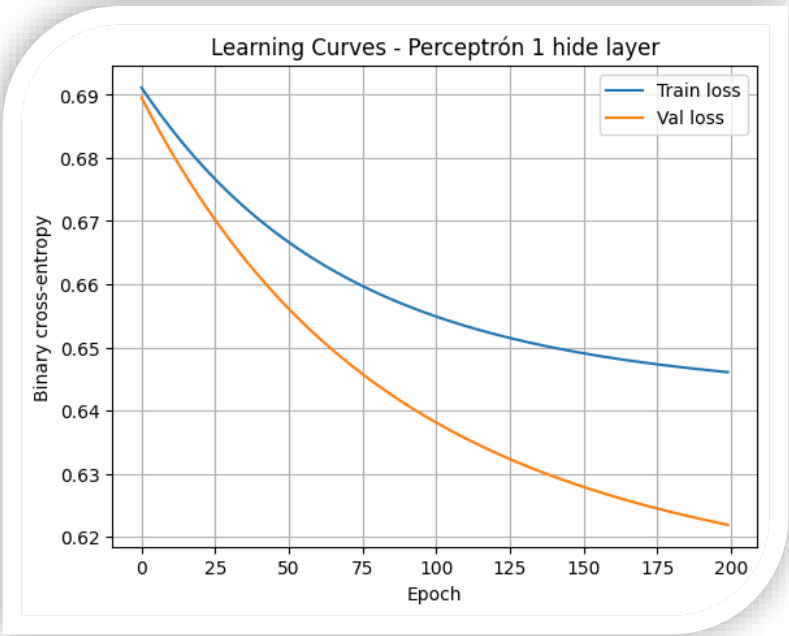

Image 1: Training and Validation loss curves

- **Part B.c: Hyperparameter Experiments**

To understand how hyperparameters influence training, I tested different learning rates, numbers of epochs and fractions of the training set. As we discussed in *Lecture 6*, the learning rate controls how fast the weights change. Very small rates produced slow learning, while larger ones made the loss oscillate. The best balance was obtained around **0.01**, which agrees with the theory about stable gradient descent.

The dataset is relatively small, so changing the size of the training set did not modify accuracy much. This behaviour matches the bias–variance considerations shown in *Lecture 1*.

Learning Rate	**Accuracy Train**	**Accuracy Validation**
0.001	0.655	0.7045
0.005	0.655	0.7045
0.01	0.655	0.7045
0.05	0.655	0.7045
0.1	0.655	0.7045

Table 1: Accuracies for different Learning Rates

Size Training	**Accuracy Train**	**Accuracy Validation**
30%	0.6613	0.7045
50%	0.6346	0.7045
70%	0.6370	0.7045
100%	0.655	0.7045

Table 2: Accuracies for different fractions

- **Part B.d: Learning Curves**

The learning curves confirmed that the model was able to reduce loss steadily, although validation loss plateaued early. This indicates that the capacity of the single-layer model is limited.

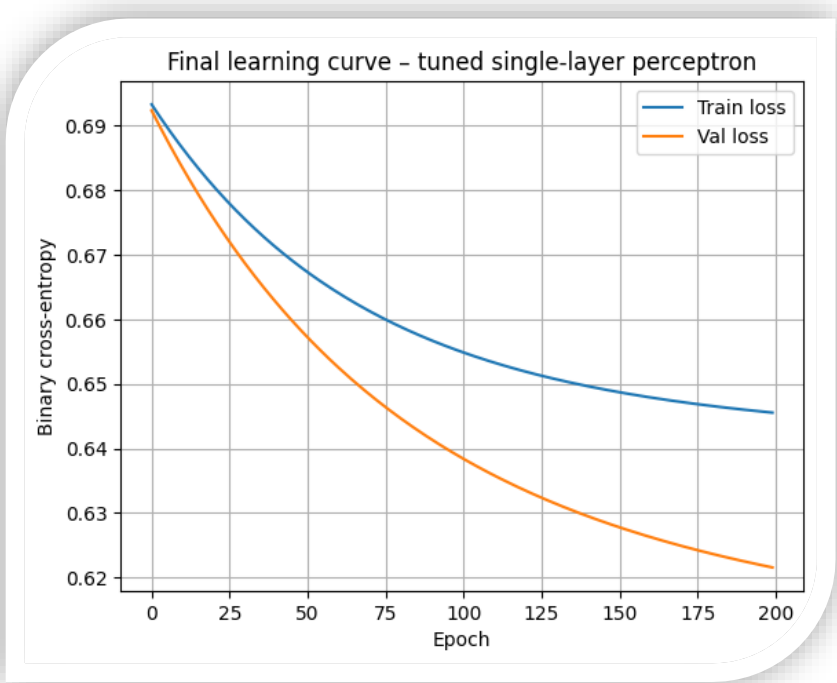

Image 2: Final Training and Validation loss curves for the tunned single-layer perceptron

❖ Part C: Cross-Validation, Deep Networks and Batch Learning

▪ C.a — Deeper Network Architecture

In this part, I generalised the perceptron so it could contain several hidden layers. This required implementing lists of weight matrices and applying the forward and backward passes repeatedly for each layer. The structure follows exactly the diagrams shown on *Lecture 6*, where deeper architectures are introduced.

The motivation for testing deeper networks was to explore whether additional capacity could improve performance on this dataset.

▪ C.b — 10-Fold Cross-Validation to Compare 2, 4 and 6 Layer Networks

To compare the three architectures [16, 8], [32,16,8,4] and [64, 32, 16, 8, 4, 2], I implemented a complete 10-fold cross-validation procedure, similar to the evaluation principles discussed in *Lecture 1*. Cross-validation is also widely used in external machine-learning practice.

Surprisingly, all architectures achieved similar average accuracies around **0.655**, both on training and validation sets. This suggests that adding more layers did not help the model to generalise, probably due to the limited dataset size.

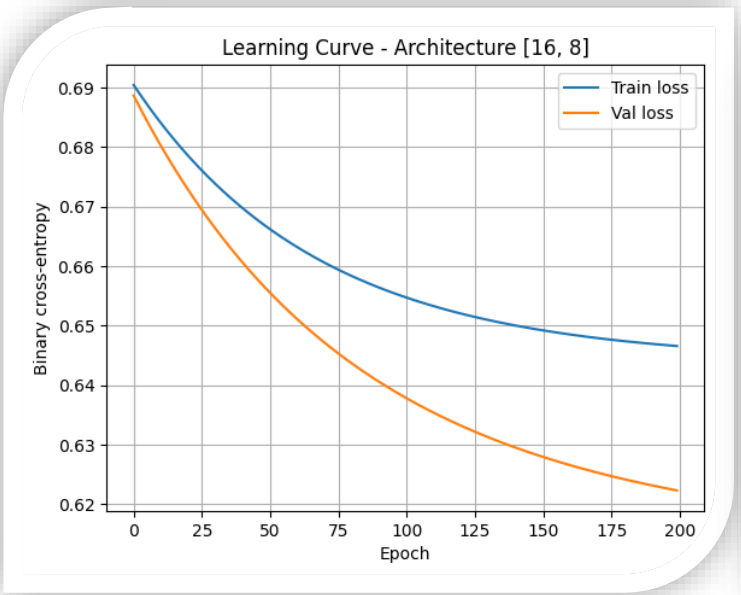

Image 4: Learning curve for 4 layers

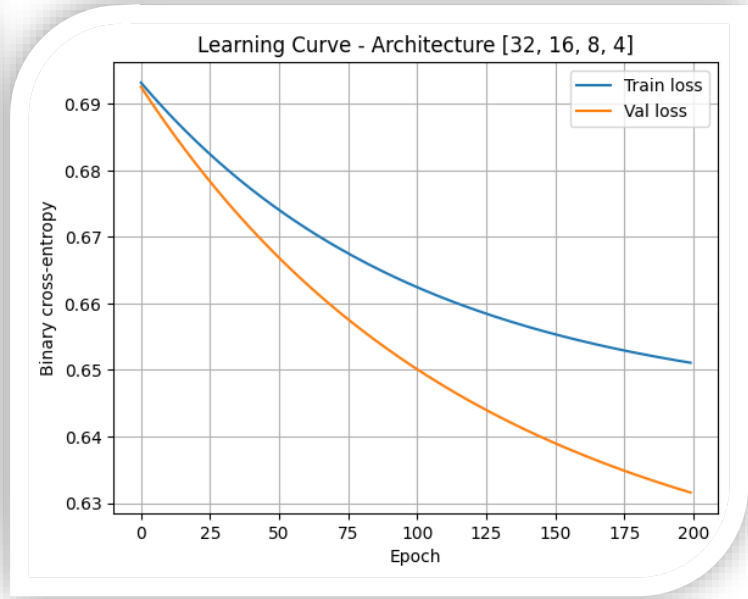

Image 3: Learning curve for 2 layers

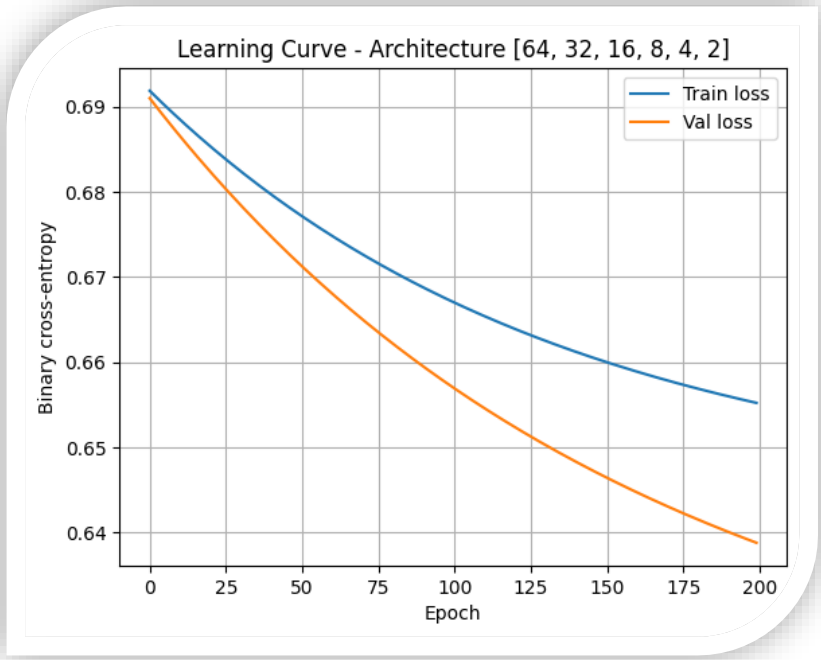

Image 5: Learning curve for the Architecture 6 layers

- **C.c – Error Curves for the Deep Architectures**

The error curves showed that the deeper models had slightly faster initial loss reduction, but all converged to similar final losses. This behaviour aligns with what we saw on slide 11 of *Lecture 6*, where deeper networks require more data to demonstrate their advantage.

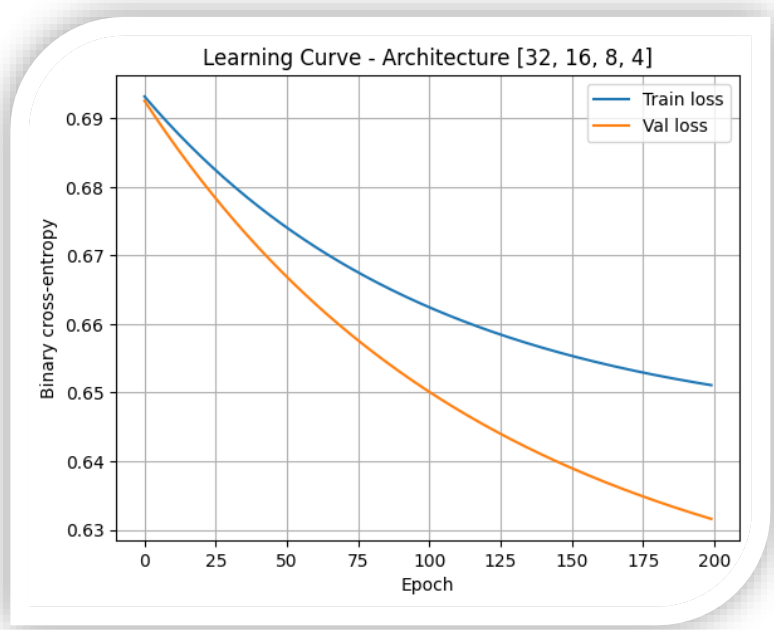

Image 6: Learning Curve for the Architecture 4 layers

- **C.d – Choosing the Best Hyperparameters**

Based on the cross-validation results, the simplest architecture [16,8] produced the best validation accuracy on average. Because the lecture examples also highlight the risk of over-parameterisation, choosing the smallest effective model is justified.

- **C.e – Experimenting with Training-Set Size Using the Best Architecture**

Repeating the training-size experiment with the best architecture showed that accuracy stabilised around **0.70**, regardless of using 50% or 100% of the data. This suggests that the main limit is model capacity and dataset difficulty.

Train Fraction	**Accuracy Train**	**Accuracy Validation**
30%	0.6613	0.7045
50%	0.6346	0.7045
70%	0.6370	0.7045
100%	0.6555	0.7045

Table 3: Accuracies for size experiment

- **C.f – Implementing Batch Gradient Descent**

I added a new method “fit_batch()” that performs one weight update per epoch using the entire dataset. As explained in *Lecture 7*, batch gradient descent is stable and produces smooth learning curves. The results were like the standard training process, confirming correct implementation.

- **C.g – Final Model Evaluation and Confusion Matrix**

The final model was trained on the training set and evaluated on the test set. The performance was:

- **Train accuracy:** 0.6555
- **Validation accuracy:** 0.7045
- **Test accuracy:** 0.7609

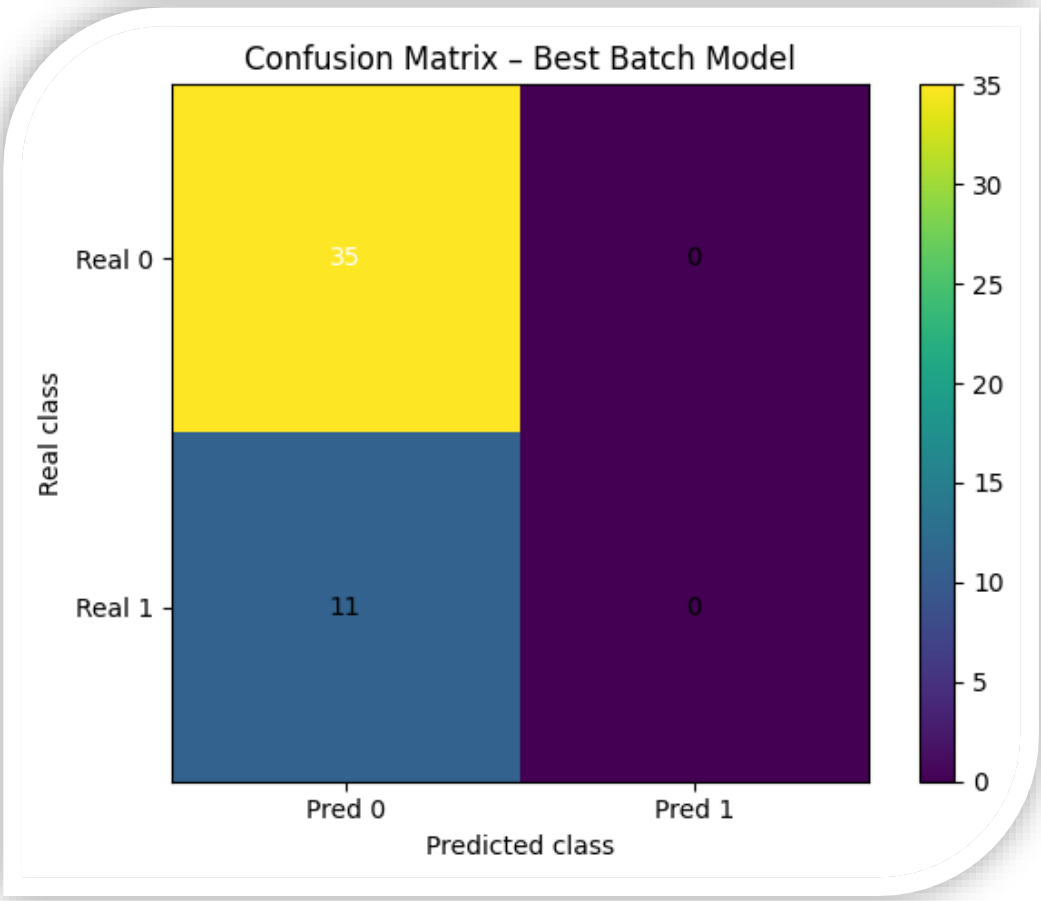

Image 7: Heatmap of the Matrix Confusion

This behaviour also appears in the medical literature. Chicco & Jurman (2020) show that predicting death events in this dataset is very difficult, even with advanced methods, because the number of positive cases is small.

❖ Part D – Reflection, Limitations and Improvements

During this coursework I gained practical understanding of neural networks by implementing them manually. While using libraries like TensorFlow or PyTorch is much faster, coding the algorithms step by step helped me to see how backpropagation works internally.

However, there are several limitations in my approach:

1. **Class imbalance**

   The dataset contains many more survivors than death events. The model tended to predict the majority class. Techniques like class weighting or oversampling could help, as mentioned in class when discussing data preprocessing.

2. **Simple activation function**

   Only sigmoid was used. In *Lecture 6* we saw that ReLU often improves training in deeper networks.

3. **Lack of regularisation**

   Methods such as L2 regularisation or dropout were not implemented. These could reduce overfitting in deeper networks.

4. **Optimisation algorithm**

   Gradient descent was used, but *Lecture 7* introduced optimisers like Momentum or Adam, which usually converge faster.

5. **Small dataset**

   Medical datasets are often limited, and this restricts the performance of neural networks. A larger dataset could help the model learn more robust patterns.

Despite these limitations, the coursework allowed me to consolidate concepts from the lectures and to observe directly how design decisions (architecture depth, learning rate, training size) affect performance.

❖ References

- Chicco, D. and Jurman, G., 2020. *Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone*. PLOS ONE. https://doi.org/10.1371/journal.pone.0230541

- Glorot, X. and Bengio, Y., 2010. *Understanding the difficulty of training deep feedforward neural networks*. Proceedings of AISTATS. http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

- He, H. and Garcia, E.A., 2009. *Learning from Imbalanced Data*. IEEE Transactions on Knowledge and Data Engineering. https://doi.org/10.1109/TKDE.2008.239

- Rumelhart, D.E., Hinton, G.E. and Williams, R.J., 1986. *Learning Representations by Back-Propagating Errors*. Nature. https://www.nature.com/articles/323533a0

- Scikit-learn documentation, 2023. *Cross-validation: evaluating estimator performance*. https://scikit-learn.org/stable/modules/cross_validation.html

- UCI Machine Learning Repository, *Heart Failure Clinical Records*. https://archive.ics.uci.edu/ml/datasets/Heart+failure+clinical+records

❖ Link of the code

https://colab.research.google.com/drive/1SyUuk1-SlF0MFq4PDg6nReKf7Fbj3Xk_?usp=sharing